

# Neural Collaborative Filtering

# Motivation

- The interaction between user and item features, they still resorted to matrix factorization and applied an inner product on the latent features of users and items.
- By replacing the inner product with a **neural architecture** that can learn an arbitrary function from data, we present a general framework named NCF, short for Neural network based Collaborative Filtering.
- NCF is generic and can express and generalize matrix factorization under its framework

# Motivation

- Much research effort has been devoted to enhancing MF, such as integrating it with **neighbor-based models** [21], combining it with topic models of item content [38], and extending it to factorization machines [26] for a generic modelling of features.
- Despite the effectiveness of MF for collaborative filtering, it is well-known that its performance can be hindered by the simple choice of the interaction function — **inner product**.
- The inner product, which simply combines the multiplication of latent features linearly, may not be sufficient to capture the complex structure of user interaction data.

# Motivation

- The neural network has been proven to be capable of approximating any **continuous function** [17], and more recently **deep neural networks** (DNNs) have been found to be effective in several domains, ranging from computer vision, speech recognition, to text processing [5, 10, 15, 47].

# Mathematical Formulation

- Let  $M$  and  $N$  denote the number of **users** and **items**, respectively.
- We define the user–item interaction matrix  $\mathbf{Y} \in \mathbb{R}^{M \times N}$  from users' **implicit feedback** as,

$$y_{ui} = \begin{cases} 1, & \text{if interaction (user } u, \text{ item } i) \text{ is observed;} \\ 0, & \text{otherwise.} \end{cases}$$

Here a value of 1 for  $y_{ui}$  indicates that there is an interaction between user  $u$  and item  $i$ ; however, it does not mean  $u$  actually likes  $i$ .

Similarly, a value of 0 does not necessarily mean  $u$  does not like  $i$ , it can be that the user is not aware of the item

# Mathematical Formulation

- The recommendation problem with implicit feedback is formulated as the problem of estimating the scores of unobserved entries in  $Y$ , which are used for ranking the items.
- Model-based approaches assume that data can be generated (or described) by an underlying model.
- Formally, they can be abstracted as learning  $y_{ui} = f(u, i | \Theta)$ , where  $y_{ui}$  denotes the predicted score of interaction  $y_{ui}$ ,  $\Theta$  denotes model parameters, and  $f$  denotes the function that maps model parameters to the predicted score (which we term as an interaction function).

# Mathematical Formulation

- To **estimate parameters**  $\Theta$ , existing approaches generally follow the machine learning paradigm that optimizes an objective function.
- Two types of objective functions are most commonly used in literature — **pointwise loss** [14, 19] and **pairwise loss** [27, 33].
- Pointwise learning usually follow a regression framework by minimizing the squared loss between  $y_{ui}$  and its target value  $y_{ui}$
- For pairwise learning [27, 44], the idea is that observed entries should be ranked higher than the unobserved ones. As such, instead of minimizing the loss between  $y_{ui}$  and  $y_{ui}$ , pairwise learning maximizes the margin between observed entry  $y_{ui}$  and unobserved entry  $y_{uj}$ .

Given a query  $q$  and a set of documents  $D = (d_1, \dots, d_n)$ :

### Pointwise

**Input:** Single candidate  
 $x = (q, d_i)$



**Loss:** How accurate is the predicted score  $s_i \approx y_i$ ?



**Solution:** Transform task into Regression.

### Pairwise

**Input:** Pair of candidates  
 $x_i = (q, d_i)$  and  $x_j = (q, d_j)$



**Loss:** If  $y_i > y_j$ , then are the predicted scores  $s_i > s_j$ ?



**Solution:** Transform task into Binary Classification.

### Listwise

**Input:** Whole list of candidates  
 $x_1 = (q, d_1) \dots x_n = (q, d_n)$



**Loss:** Takes into account position of all retrieved docs.



**Solution:** Incorporate evaluation metrics (e.g. DCG) into loss.

**1. Pointwise loss** follows a regression framework by minimizing the squared loss between predicted score  $\hat{y}(u,i)$  and target value  $y(u,i)$ .

In order to account for negative feedback either all unobserved entries are considered as negative examples, or some unobserved entries are sampled to be negative instances.

**2. Pairwise loss** aims to rank observed entries higher than the unobserved ones. It achieves this by maximizing the margin between observed entries and unobserved entries.

To summarise, pairwise loss maximizes the margin between observed and unobserved entries in contrast to pointwise loss which aims to minimize the loss between predicted and target score.

# Matrix Factorization

MF estimates an interaction  $y_{ui}$  as the inner product of  $\mathbf{p}_u$  and  $\mathbf{q}_i$

$$\hat{y}_{ui} = f(u, i | \mathbf{p}_u, \mathbf{q}_i) = \mathbf{p}_u^T \mathbf{q}_i = \sum_{k=1}^K p_{uk} q_{ik},$$

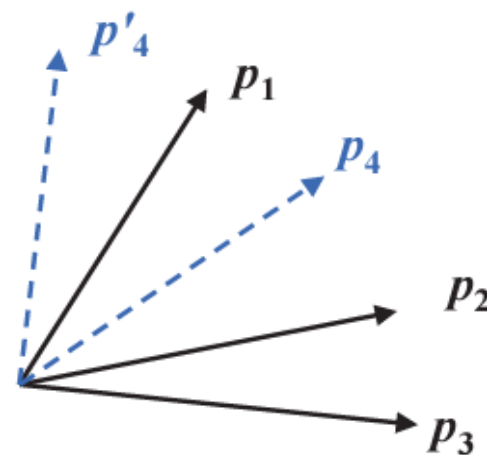
# Problem with MF

|       | $i_1$ | $i_2$ | $i_3$ | $i_4$ | $i_5$ |
|-------|-------|-------|-------|-------|-------|
| $u_1$ | 1     | 1     | 1     | 0     | 1     |
| $u_2$ | 0     | 1     | 1     | 0     | 0     |
| $u_3$ | 0     | 1     | 1     | 1     | 0     |
| $u_4$ | 1     | 0     | 1     | 1     | 1     |

← items →

↑ users

(a) user-item matrix



(b) user latent space

Figure 1: An example illustrates MF's limitation. From data matrix (a),  $u_4$  is most similar to  $u_1$ , followed by  $u_3$ , and lastly  $u_2$ . However in the latent space (b), placing  $p_4$  closest to  $p_1$  makes  $p_4$  closer to  $p_2$  than  $p_3$ , incurring a large ranking loss.

# Neural collaborative filtering framework

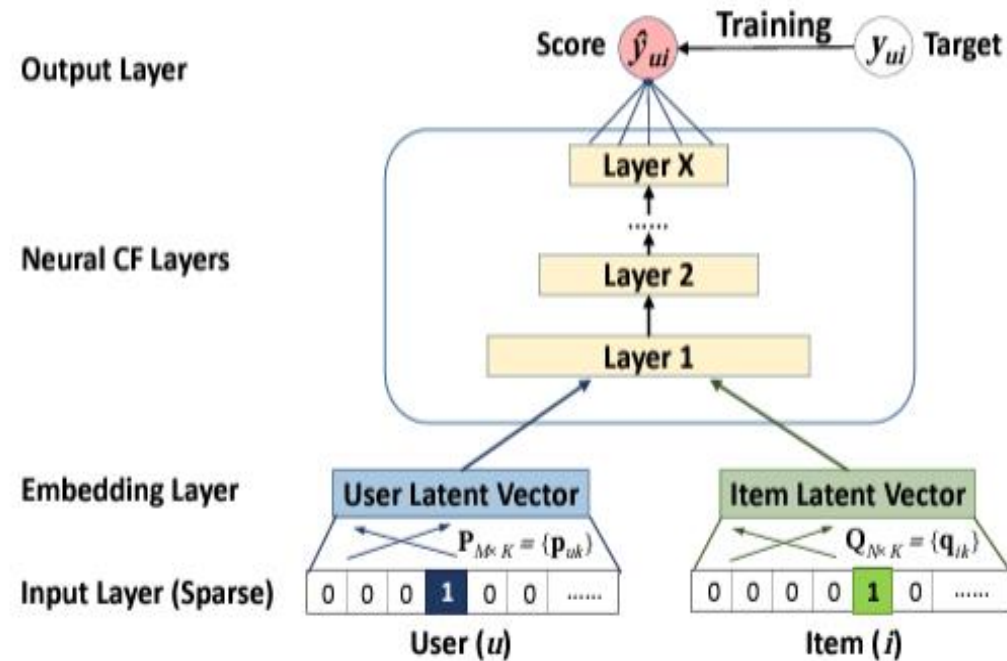


Figure 2: Neural collaborative filtering framework

Adopt a multi-layer representation to model a user–item interaction  $y_{ui}$  as shown in Figure 2, where the output of one layer serves as the input of the next one.

# Neural collaborative filtering framework

- The bottom input layer consists of two feature vectors  $\mathbf{v}_u^U$  and  $\mathbf{v}_i^I$  that describe user  $u$  and item  $i$ , respectively; they can be customized to support a wide range of modelling of users and items.
- Since this work focuses on the pure collaborative filtering setting, we use only the **identity of a user and an item as the input feature**, transforming it to a binarized sparse vector with **one-hot encoding**.
- Note that with such a generic feature representation for inputs, our method can be easily adjusted to address the cold-start problem by using content features to represent users and items.

# Neural collaborative filtering framework

- Above the input layer is the embedding layer; it is a fully connected layer that projects the sparse representation to a dense vector.
- The obtained user (item) embedding can be seen as the latent vector for user (item) in the context of latent factor model.
- The user embedding and item embedding are then fed into a multi-layer neural architecture, which we term as neural collaborative filtering layers, to map the latent vectors to prediction score.
- Each layer of the neural CF layers can be customized to discover certain latent structures of user–item interactions.
- The dimension of the last hidden layer  $X$  determines the model's capability.
- The final output layer is the predicted score  $y_{ui}$ , and training is performed by minimizing the pointwise loss between  $y_{ui}$  and its target value  $y_{ui}$ .
- **We note that another way to train the model is by performing pairwise learning, such as using the Bayesian Personalized Ranking [27] and margin-based loss [33].**

# Neural collaborative filtering framework

- We now formulate the NCF's predictive model as,

$$\hat{y}_{ui} = f(\mathbf{P}^T \mathbf{v}_u^U, \mathbf{Q}^T \mathbf{v}_i^I | \mathbf{P}, \mathbf{Q}, \Theta_f), \quad (3)$$

where  $\mathbf{P} \in \mathbb{R}^{M \times K}$  and  $\mathbf{Q} \in \mathbb{R}^{N \times K}$ , denoting the latent factor matrix for users and items, respectively; and  $\Theta_f$  denotes the model parameters of the interaction function  $f$ . Since the function  $f$  is defined as a multi-layer neural network, it can be formulated as

$$f(\mathbf{P}^T \mathbf{v}_u^U, \mathbf{Q}^T \mathbf{v}_i^I) = \phi_{out}(\phi_X(\dots \phi_2(\phi_1(\mathbf{P}^T \mathbf{v}_u^U, \mathbf{Q}^T \mathbf{v}_i^I))\dots)), \quad (4)$$

where  $\phi_{out}$  and  $\phi_x$  respectively denote the mapping function for the output layer and  $x$ -th neural collaborative filtering (CF) layer, and there are  $X$  neural CF layers in total.

# Learning NCF

- To learn model parameters, existing pointwise methods [14, 39] largely perform a regression with squared loss,

$$L_{sqr} = \sum_{(u,i) \in \mathcal{Y} \cup \mathcal{Y}^-} w_{ui} (y_{ui} - \hat{y}_{ui})^2,$$

where  $\mathcal{Y}$  denotes the set of observed interactions in  $Y$ , and  $\mathcal{Y}^-$  denotes the set of negative instances, which can be all (or sampled from) unobserved interactions; and  $w_{ui}$  is a **hyperparameter denoting the weight of training instance (u,i)**.

For implicit data, the target value  $y_{ui}$  is a binarized 1 or 0 denoting whether  $u$  has interacted with  $i$ . In what follows, we present a probabilistic approach for learning the pointwise NCF that pays special attention to the binary property of implicit data.

# Learning NCF

- Considering the one-class nature of implicit feedback, we can view the value of  $y_{ui}$  as a label-1 means item  $i$  is relevant to  $u$ , and 0 otherwise.
- The prediction score  $y_{ui}$  then represents how likely  $i$  is relevant to  $u$ .
- To endow NCF with such a probabilistic explanation, we need to constrain the output  $y_{ui}$  in the range of  $[0,1]$ , which can be easily achieved by using a probabilistic function (e.g., the Logistic or Probit function) as the activation function for the output layer  $\phi_{\text{out}}$ .
- With the above settings, we then define the likelihood function as,

$$p(\mathcal{Y}, \mathcal{Y}^- | \mathbf{P}, \mathbf{Q}, \Theta_f) = \prod_{(u,i) \in \mathcal{Y}} \hat{y}_{ui} \prod_{(u,j) \in \mathcal{Y}^-} (1 - \hat{y}_{uj}). \quad (6)$$

Taking the negative logarithm of the likelihood, we reach

$$\begin{aligned} L &= - \sum_{(u,i) \in \mathcal{Y}} \log \hat{y}_{ui} - \sum_{(u,j) \in \mathcal{Y}^-} \log(1 - \hat{y}_{uj}) \\ &= - \sum_{(u,i) \in \mathcal{Y} \cup \mathcal{Y}^-} y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log(1 - \hat{y}_{ui}). \end{aligned} \quad (7)$$

This is the objective function to minimize for the NCF methods, and its optimization can be done by performing stochastic gradient descent (SGD)

Careful readers might have realized that it is the same as the binary cross-entropy loss, also known as log loss.

# Generalized Matrix Factorization (GMF)

- MF can be interpreted as a special case of our NCF framework.
- Due to the one-hot encoding of user (item) ID of the input layer, the obtained embedding vector can be seen as the **latent vector** of user (item).
- Let the user latent vector  $\mathbf{p}_u$  be  $\mathbf{P}^T \mathbf{V}_u^U$  and item latent vector  $\mathbf{q}_i$  be  $\mathbf{Q}^T \mathbf{V}_i^I$
- Mapping function of the first neural CF layer can be defined as,

$$\phi_1(\mathbf{p}_u, \mathbf{q}_i) = \mathbf{p}_u \odot \mathbf{q}_i, \quad (8)$$

where  $\odot$  denotes the element-wise product of vectors. We then project the vector to the output layer:

$$\hat{y}_{ui} = a_{out}(\mathbf{h}^T (\mathbf{p}_u \odot \mathbf{q}_i)), \quad (9)$$

where  $a_{out}$  and  $\mathbf{h}$  denote the activation function and edge weights of the output layer, respectively. Intuitively, if we use an identity function for  $a_{out}$  and enforce  $\mathbf{h}$  to be a uniform vector of 1, we can exactly recover the MF model.

# Generalized Matrix Factorization (GMF)

- Under the NCF framework, MF can be easily generalized and extended. For example, if we allow  $h$  to be from data without the uniform constraint, it will result in a variant of MF that allows varying importance of latent dimensions.
- And if we use a non-linear function for  $a_{\text{out}}$ , it will generalize MF to a **non-linear setting** which might be more expressive than the linear MF model.
- In this work, we implement a generalized version of MF under NCF that uses the **sigmoid function**  $\sigma(x) = 1/(1 + e^{-x})$  as  $a_{\text{out}}$  and learns  $h$  from data with the log loss.
- We term it as GMF, short for **Generalized Matrix Factorization**.

# Multi-Layer Perceptron (MLP)

- NCF adopts two pathways to model users and items, it is intuitive to **combine the features of two** pathways by concatenating them.
- However, simply a **vector concatenation** does not account for any interactions between user and item latent features, which is insufficient for modelling the collaborative filtering effect.
- To address this issue, we propose to add hidden layers on the concatenated vector, using a standard **MLP** to learn the interaction between user and item latent features.
- In this sense, we can endow the model a large level of flexibility and non-linearity to learn the interactions between  **$\mathbf{p}_u$  and  $\mathbf{q}_i$** , rather than the way of GMF that uses only a fixed element-wise product on them.

# Multi-Layer Perceptron (MLP)

More precisely, the MLP model under our NCF framework is defined as,

$$\begin{aligned} \mathbf{z}_1 &= \phi_1(\mathbf{p}_u, \mathbf{q}_i) = \begin{bmatrix} \mathbf{p}_u \\ \mathbf{q}_i \end{bmatrix}, \\ \phi_2(\mathbf{z}_1) &= a_2(\mathbf{W}_2^T \mathbf{z}_1 + \mathbf{b}_2), \\ &\dots\dots \\ \phi_L(\mathbf{z}_{L-1}) &= a_L(\mathbf{W}_L^T \mathbf{z}_{L-1} + \mathbf{b}_L), \\ \hat{y}_{ui} &= \sigma(\mathbf{h}^T \phi_L(\mathbf{z}_{L-1})), \end{aligned}$$

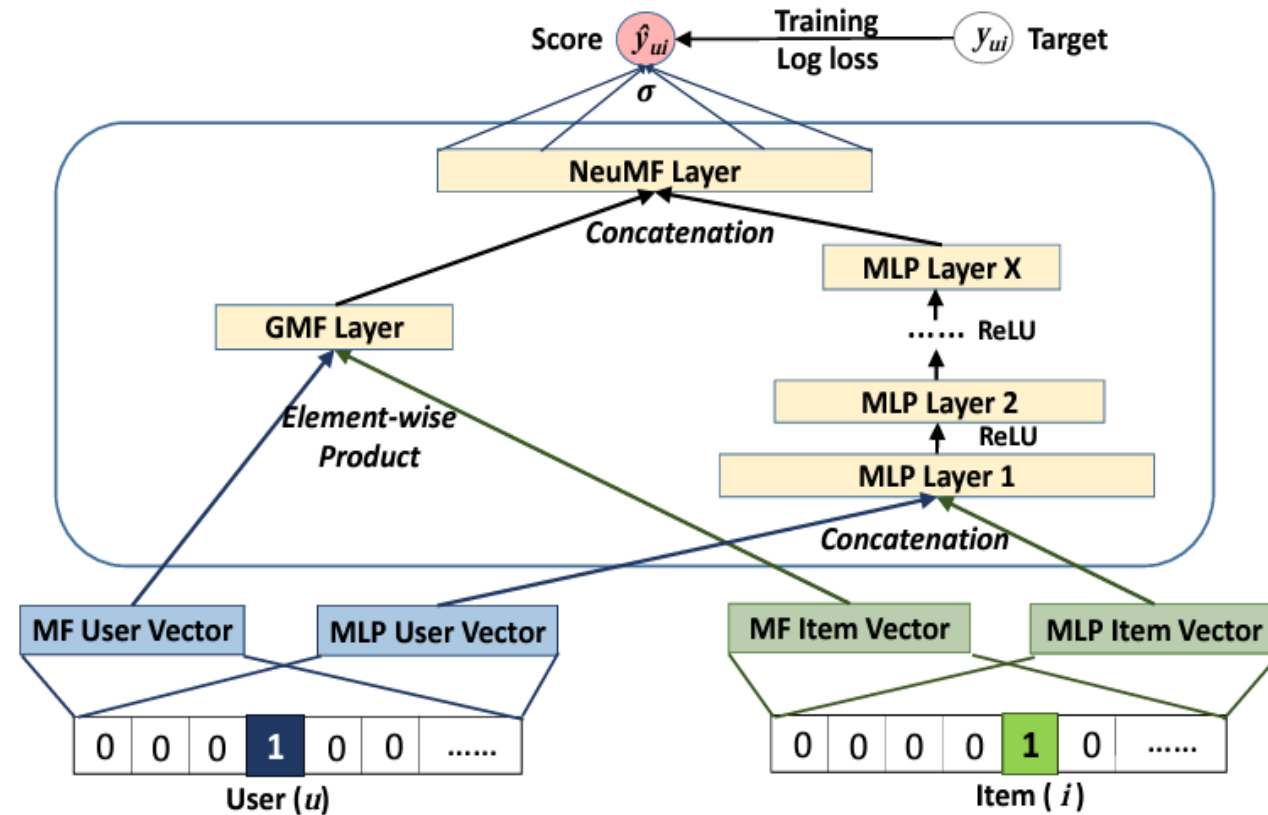
where  $\mathbf{W}_x$ ,  $\mathbf{b}_x$ , and  $a_x$  denote the weight matrix, bias vector, and activation function for the  $x$ -th layer's perceptron, respectively

# Fusion of GMF and MLP

A straightforward solution is to let GMF and MLP share the same embedding layer, and then combine the outputs of their interaction functions.

This way shares a similar spirit with the well-known Neural Tensor Network (NTN) [33]. Specifically, the model for combining GMF with a one-layer MLP can be formulated as,

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T a(\mathbf{p}_u \odot \mathbf{q}_i + \mathbf{W} \begin{bmatrix} \mathbf{p}_u \\ \mathbf{q}_i \end{bmatrix} + \mathbf{b})).$$



However, sharing embeddings of GMF and MLP might limit the performance of the fused model

# Fusion of GMF and MLP

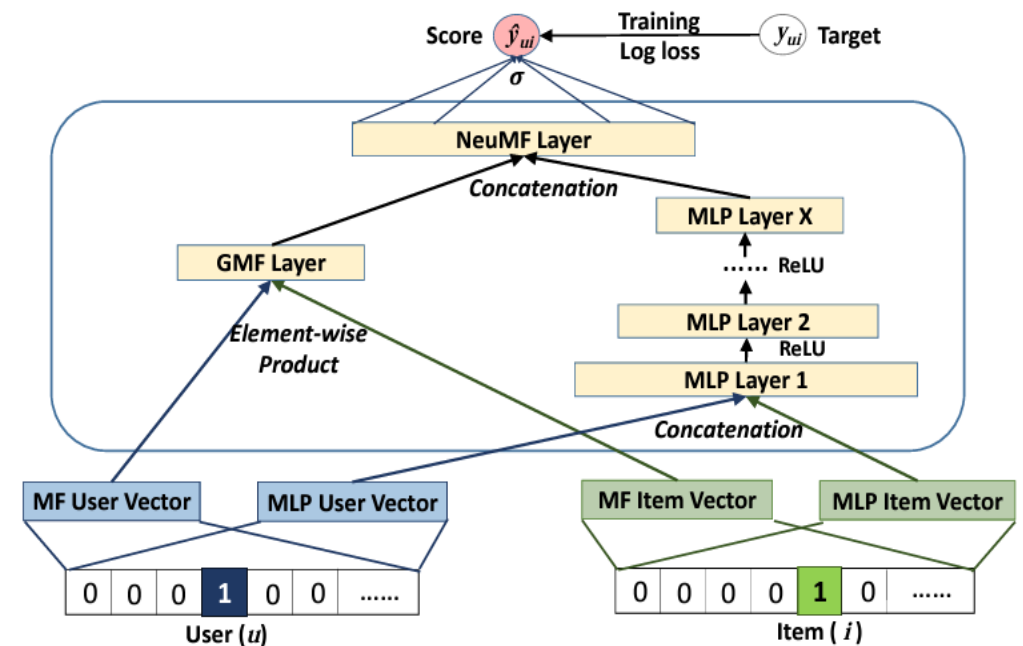
- To provide more flexibility to the fused model, we allow GMF and MLP to learn separate embeddings, and combine the two models by concatenating their last hidden layer.

$$\phi^{GMF} = \mathbf{p}_u^G \odot \mathbf{q}_i^G,$$

$$\phi^{MLP} = a_L(\mathbf{W}_L^T(a_{L-1}(\dots a_2(\mathbf{W}_2^T \begin{bmatrix} \mathbf{p}_u^M \\ \mathbf{q}_i^M \end{bmatrix} + \mathbf{b}_2)\dots)) + \mathbf{b}_L)$$

$$\hat{y}_{ui} = \sigma(\mathbf{h}^T \begin{bmatrix} \phi^{GMF} \\ \phi^{MLP} \end{bmatrix}),$$

where  $\mathbf{p}_u^G$  and  $\mathbf{p}_u^M$  denote the user embedding for GMF and MLP parts, respectively; and similar notations of  $\mathbf{q}_i^G$  and  $\mathbf{q}_i^M$  for item embeddings.



This model combines the linearity of MF and non-linearity of DNNs for modelling user-item latent structures.

# Valid Questions

- Do our proposed NCF methods outperform the state of-the-art implicit collaborative filtering methods?
- How does our proposed optimization framework (log loss with negative sampling) work for the recommendation task?
- Are deeper layers of hidden units helpful for learning from user–item interaction data?

# Datasets

- Datasets. We experimented with two publicly accessible datasets: MovieLens<sup>4</sup> and Pinterest<sup>5</sup>.
- The characteristics of the two datasets are summarized in Table 1.

**Table 1: Statistics of the evaluation datasets.**

| <b>Dataset</b> | <b>Interaction#</b> | <b>Item#</b> | <b>User#</b> | <b>Sparsity</b> |
|----------------|---------------------|--------------|--------------|-----------------|
| MovieLens      | 1,000,209           | 3,706        | 6,040        | 95.53%          |
| Pinterest      | 1,500,809           | 9,916        | 55,187       | 99.73%          |

<sup>4</sup><http://grouplens.org/datasets/movielens/1m/>

<sup>5</sup><https://sites.google.com/site/xueatalphabeta/academic-projects>

# Movie Lense Dataset

- This movie rating dataset has been widely used to evaluate collaborative filtering algorithms.
- We used the version containing one million ratings, where each user has at least 20 ratings.
- While it is an explicit feedback data, we have intentionally chosen it to investigate the performance of learning from the implicit signal [21] of explicit feedback.
- To this end, we transformed it into implicit data, where each entry is marked as 0 or 1 indicating whether the user has rated the item.

# Pinterest Dataset

- This implicit feedback data is constructed by [8] for evaluating content-based image recommendation.
- The original data is very large but highly sparse. For example, over 20% of users have only one pin, making it difficult to evaluate collaborative filtering algorithms.
- As such, we filtered the dataset in the same way as the MovieLens data that retained only users with at least 20 interactions (pins).
- This results in a subset of the data that contains 55,187 users and 1,500,809 interactions. Each interaction denotes whether the user has pinned the image to her own board.

# Evaluation Protocols

- To evaluate the performance of item recommendation, we adopted the leave-one-out evaluation, which has been widely used in literature.
- For each user, we held-out her latest interaction as the test set and utilized the remaining data for training.
- Since it is too time-consuming to rank all items for every user during evaluation, we followed the common strategy [6, 21] that randomly samples 100 items that are not interacted by the user, ranking the test item among the 100 items.
- The performance of a ranked list is judged by Hit Ratio (HR) and Normalized Discounted Cumulative Gain (NDCG) [11].

# Baselines

- ItemPop. Items are ranked by their popularity judged by the number of interactions. This is a non-personalized method to benchmark the recommendation performance.
- ItemKNN [31]. This is the standard item-based collaborative filtering method. We followed the setting of [19] to adapt it for implicit data.
- BPR [27]. This method optimizes the MF model of Equation 2 with a pairwise ranking loss, which is tailored to learn from implicit feedback. It is a highly competitive baseline for item recommendation. We used a fixed learning rate, varying it and reporting the best performance.
- eALS [14]. This is a state-of-the-art MF method for item recommendation. It optimizes the squared loss of Equation 5, treating all unobserved interactions as negative instances and weighting them non-uniformly by the item popularity. Since eALS shows superior performance over the uniform-weighting method WMF [19], we do not further report WMF's performance.

# Parameters

- All NCF models are learnt by optimizing the log loss of Equation 7, where we sampled four negative instances per positive instance.
- For NCF models that are trained from scratch, we randomly initialized model parameters with a Gaussian distribution (with a mean of 0 and standard deviation of 0.01), optimizing the model with mini-batch Adam [20].
- We tested the batch size of [128,256,512,1024], and the learning rate of [0.0001 ,0.0005, 0.001, 0.005].
- Since the last hidden layer of NCF determines the model capability, we term it as predictive factors and evaluated the factors of [8,16,32,64].
- It is worth noting that large factors may cause overfitting and degrade the performance. Without special mention, we employed three hidden layers for MLP; for example, if the size of predictive factors is 8, then the architecture of the neural CF layers is  $32 \rightarrow 16 \rightarrow 8$ , and the embedding size is 16. For the NeuMF with pre-training,  $\alpha$  was set to 0.5, allowing the pre-trained GMF and MLP to contribute equally to NeuMF's initialization

# Performance Comparison (RQ1)